

[LEARN REACT](#) > [ADDING INTERACTIVITY](#) >

Updating Arrays in State

Arrays are mutable in JavaScript, but you should treat them as immutable when you store them in state. Just like with objects, when you want to update an array stored in state, you need to create a new one (or make a copy of an existing one), and then set state to use the new array.

You will learn

- How to add, remove, or change items in an array in React state
- How to update an object inside of an array
- How to make array copying less repetitive with Immer

Updating arrays without mutation

In JavaScript, arrays are just another kind of object. [Like with objects](#), you should treat **arrays in React state as read-only**. This means that you shouldn't reassign items inside an array like `arr[0] = 'bird'`, and you also shouldn't use methods that mutate the array, such as `push()` and `pop()`.

Instead, every time you want to update an array, you'll want to pass a *new* array to your state setting function. To do that, you can create a new array from the original array in your state by calling its non-mutating methods like `filter()` and `map()`. Then you can set your state to the resulting new array.

Here is a reference table of common array operations. When dealing with arrays inside React state, you will need to avoid the methods in the left column, and instead prefer

the methods in the right column:

	avoid (mutates the array)	prefer (returns a new array)
adding	<code>push</code> , <code>unshift</code>	<code>concat</code> , <code>[...arr]</code> spread syntax (example)
removing	<code>pop</code> , <code>shift</code> , <code>splice</code>	<code>filter</code> , <code>slice</code> (example)
replacing	<code>splice</code> , <code>arr[i] = ...</code> assignment	<code>map</code> (example)
sorting	<code>reverse</code> , <code>sort</code>	copy the array first (example)

Alternatively, you can [use Immer](#) which lets you use methods from both columns.

Pitfall

Unfortunately, `slice` and `splice` are named similarly but are very different:

- `slice` lets you copy an array or a part of it.
- `splice` **mutates** the array (to insert or delete items).

In React, you will be using `slice` (no `p`!) a lot more often because you don't want to mutate objects or arrays in state. [Updating Objects](#) explains what mutation is and why it's not recommended for state.

Adding to an array

`push()` will mutate an array, which you don't want:

App.js

[Download](#) [Reset](#) [Share](#)

```
1 import { useState } from 'react';
2
3 let nextId = 0;
4
5 export default function List() {
6   const [name, setName] = useState('');
7   const [artists, setArtists] = useState([]);
8
9   return (
10     <>
11       <h1>Inspiring sculptors:</h1>
12       <input
```

[Show more](#)

Instead, create a *new* array which contains the existing items *and* a new item at the end. There are multiple ways to do this, but the easiest one is to use the `... array spread` syntax:

```
setArtists( // Replace the state
```

```
[ // with a new array
  ...artists, // that contains all the old items
  { id: nextId++, name: name } // and one new item at the end
]
);
```

Now it works correctly:

App.js

[Download](#) [Reset](#) [Share](#)

```
11 <div>Inspiring sculptors:</div>
12 <input
13   value={name}
14   onChange={e => setName(e.target.value)}
15 />
16 <button onClick={() => {
17   setArtists([
18     ...artists,
19     { id: nextId++, name: name }
20   ]);
21 >Add</button>
22 <ul>
23   {artists.map(artist => (
```

▼ Show more

The array spread syntax also lets you prepend an item by placing it *before* the original `...artists`:

```
setArtists([
  { id: nextId++, name: name },
  ...artists // Put old items at the end
]);
```

In this way, spread can do the job of both `push()` by adding to the end of an array and `unshift()` by adding to the beginning of an array. Try it in the sandbox above!

Removing from an array

The easiest way to remove an item from an array is to *filter it out*. In other words, you will produce a new array that will not contain that item. To do this, use the `filter` method, for example:

App.js

Download Reset

```
30 {artists.map(artist => (  
31   <li key={artist.id}>  
32     {artist.name}{' '}  
33     <button onClick={() => {  
34       setArtists(  
35         artists.filter(a =>  
36           a.id !== artist.id  
37         )  
38       )};  
39     }}>  
40       Delete  
41     </button>  
42   )  
43 )}
```

▼ Show more

Inspiring sculptors:

- Marta Colvin Andrade Delete
- Lamidi Olonade Fakeye Delete
- Louise Nevelson Delete

Click the “Delete” button a few times, and look at its click handler.

```
setArtists(  
  artists.filter(a => a.id !== artist.id)  
);
```

Here, `artists.filter(a => a.id !== artist.id)` means “create an array that consists of those `artists` whose IDs are different from `artist.id`”. In other words, each artist’s “Delete” button will filter *that* artist out of the array, and then request a re-render with the resulting array. Note that `filter` does not modify the original array.

Transforming an array

If you want to change some or all items of the array, you can use `map()` to create a **new** array. The function you will pass to `map` can decide what to do with each item, based on its data or its index (or both).

In this example, an array holds coordinates of two circles and a square. When you press the button, it moves only the circles down by 50 pixels. It does this by producing a new array of data using `map()`:

App.js

[Download](#) [Reset](#) [Share](#)

```
12   );  
13  
14   function handleClick() {  
15     const nextShapes = shapes.map(shape => {  
16       if (shape.type === 'square') {  
17         // No change  
18         return shape;  
19       } else {  
20         // Return a new circle 50px below  
21         return {  
22           ...shape,  
23           y: shape.y + 50,  
24         }  
25       }  
26     });  
27     setShapes(nextShapes);  
28   }  
29  
30   return (
```

▼ Show more

Move circles down!

Replacing items in an array

It is particularly common to want to replace one or more items in an array.

Assignments like `arr[0] = 'bird'` are mutating the original array, so instead you'll want to use `map` for this as well.

To replace an item, create a new array with `map`. Inside your `map` call, you will receive the item index as the second argument. Use it to decide whether to return the original item (the first argument) or something else:

App.js

[Download](#) [Reset](#) [Link](#)

```
1  import { useState } from 'react';
2
3
4  let initialCounters = [
5
6    0, 0, 0
7
8  ];
9
10
11 export default function CounterList() {
12
13   const [counters, setCounters] = useState(
14     initialCounters
15   );
16
17
18
19   function handleIncrementClick(index) {
```

▼ Show more

- 0
- 0
- 0

Inserting into an array

Sometimes, you may want to insert an item at a particular position that's neither at the beginning nor at the end. To do this, you can use the `...` array spread syntax together with the `slice()` method. The `slice()` method lets you cut a “slice” of the array. To insert an item, you will create an array that spreads the slice *before* the insertion point, then the new item, and then the rest of the original array.

In this example, the Insert button always inserts at the index `1`:

App.js

Download Reset

```
import { useState } from 'react';

let nextId = 3;
const initialArtists = [
  { id: 0, name: 'Marta Colvin Andrade' },
  { id: 1, name: 'Lamidi Olonade Fakeye'},
  { id: 2, name: 'Louise Nevelson'},
];

export default function List() {
  const [name, setName] = useState('');
  const [artists, setArtists] = useState(
```

▼ Show more

Inspiring sculptors:

- Marta Colvin Andrade
- Lamidi Olonade Fakeye
- Louise Nevelson

Making other changes to an array

There are some things you can't do with the spread syntax and non-mutating methods like `map()` and `filter()` alone. For example, you may want to reverse or sort an array. The JavaScript `reverse()` and `sort()` methods are mutating the original array, so you can't use them directly.

However, you can copy the array first, and then make changes to it.

For example:

App.js

Download Reset

```
import { useState } from 'react';

let nextId = 3;
const initialList = [
  { id: 0, title: 'Big Bellies' },
  { id: 1, title: 'Lunar Landscape' },
  { id: 2, title: 'Terracotta Army' },
];

export default function List() {
  const [list, setList] = useState(initialList);
```

▼ Show more

Here, you use the `[...list]` spread syntax to create a copy of the original array first. Now that you have a copy, you can use mutating methods like `nextList.reverse()` or `nextList.sort()`, or even assign individual items with `nextList[0] = "something"`.

However, **even if you copy an array, you can't mutate existing items *inside* of it directly**. This is because copying is shallow—the new array will contain the same items as the original one. So if you modify an object inside the copied array, you are mutating the existing state. For example, code like this is a problem.

```
const nextList = [...list];
nextList[0].seen = true; // Problem: mutates list[0]
setList(nextList);
```

Although `nextList` and `list` are two different arrays, `nextList[0]` and `list[0]` **point to the same object**. So by changing `nextList[0].seen`, you are also changing `list[0].seen`. This is a state mutation, which you should avoid! You can solve this issue in a similar way to [updating nested JavaScript objects](#)—by copying individual items you want to change instead of mutating them. Here's how.

Updating objects inside arrays

Objects are not *really* located “inside” arrays. They might appear to be “inside” in code, but each object in an array is a separate value, to which the array “points”. This is why you need to be careful when changing nested fields like `list[0]`. Another person’s artwork list may point to the same element of the array!

When updating nested state, you need to create copies from the point where you want to update, and all the way up to the top level. Let’s see how this works.

In this example, two separate artwork lists have the same initial state. They are supposed to be isolated, but because of a mutation, their state is accidentally shared, and checking a box in one list affects the other list:

App.js

[Download](#) [Reset](#) [Share](#)

```
import { useState } from 'react';

let nextId = 3;
const initialList = [
  { id: 0, title: 'Big Bellies', seen: false },
  { id: 1, title: 'Lunar Landscape', seen: false },
  { id: 2, title: 'Terracotta Army', seen: true },
];

export default function BucketList() {
  const [myList, setMyList] = useState(initialList);
  const [yourList, setYourList] = useState(
```

▼ Show more

The problem is in code like this:

```
const myNextList = [...myList];
const artwork = myNextList.find(a => a.id === artworkId);
artwork.seen = nextSeen; // Problem: mutates an existing item
setMyList(myNextList);
```

Although the `myNextList` array itself is new, the *items themselves* are the same as in the original `myList` array. So changing `artwork.seen` changes the *original* artwork item. That artwork item is also in `yourList`, which causes the bug. Bugs like this can be difficult to think about, but thankfully they disappear if you avoid mutating state.

You can use `map` to substitute an old item with its updated version without mutation.

```
setMyList(myList.map(artwork => {
  if (artwork.id === artworkId) {
    // Create a *new* object with changes
    return { ...artwork, seen: nextSeen };
  } else {
    // No changes
    return artwork;
  }
}));
```

Here, `...` is the object spread syntax used to [create a copy of an object](#).

With this approach, none of the existing state items are being mutated, and the bug is fixed:

App.js

Download Reset

```
import { useState } from 'react';

let nextId = 3;
const initialList = [
  { id: 0, title: 'Big Bellies', seen: false },
  { id: 1, title: 'Lunar Landscape', seen: false },
  { id: 2, title: 'Terracotta Army', seen: true },
];

export default function BucketList() {
  const [myList, setMyList] = useState(initialList);
  const [yourList, setYourList] = useState(
```

▼ Show more

In general, **you should only mutate objects that you have just created**. If you were inserting a *new* artwork, you could mutate it, but if you're dealing with something

that's already in state, you need to make a copy.

Write concise update logic with Immer

Updating nested arrays without mutation can get a little bit repetitive. [Just as with objects](#):

- Generally, you shouldn't need to update state more than a couple of levels deep. If your state objects are very deep, you might want to [restructure them differently](#) so that they are flat.
- If you don't want to change your state structure, you might prefer to use [Immer](#), which lets you write using the convenient but mutating syntax and takes care of producing the copies for you.

Here is the Art Bucket List example rewritten with Immer:

App.js package.json

↺ Reset ↗

```
import { useState } from 'react';
import { useImmer } from 'use-immer';

let nextId = 3;
const initialList = [
  { id: 0, title: 'Big Bellies', seen: false },
  { id: 1, title: 'Lunar Landscape', seen: false },
  { id: 2, title: 'Terracotta Army', seen: true },
];

export default function BucketList() {
  const [myList, updateMyList] = useImmer(
```

▼ Show more

Note how with Immer, **mutation** like `artwork.seen = nextSeen` is now okay:

```
updateMyTodos(draft => {  
  const artwork = draft.find(a => a.id === artworkId);  
  artwork.seen = nextSeen;  
});
```

This is because you're not mutating the *original* state, but you're mutating a special `draft` object provided by Immer. Similarly, you can apply mutating methods like `push()` and `pop()` to the content of the `draft`.

Behind the scenes, Immer always constructs the next state from scratch according to the changes that you've done to the `draft`. This keeps your event handlers very concise without ever mutating state.

Recap

- You can put arrays into state, but you can't change them.
- Instead of mutating an array, create a *new* version of it, and update the state to it.
- You can use the `[...arr, newItem]` array spread syntax to create arrays with new items.
- You can use `filter()` and `map()` to create new arrays with filtered or transformed items.

- You can use Immer to keep your code concise.

Try out some challenges

1. Update an item in the shopping cart 2. Remove an item from the shopping cart 3.



Challenge 1 of 4:

Update an item in the shopping cart

Fill in the `handleIncreaseClick` logic so that pressing "+" increases the corresponding number:

App.js

Download Reset

```
import { useState } from 'react';

const initialProducts = [{
  id: 0,
  name: 'Baklava',
  count: 1,
}, {
  id: 1,
  name: 'Cheese',
  count: 5,
}, {
  id: 2,
```

▼ Show more